

Angular - Big Picture

Views (HTML)

A view is the component's HTML template

- the visible part rendered in the browser.
- displays the component's state,
- accepts user input
- binds browser events back to component methods.

When the component state changes, Angular updates the relevant parts of the view.

Login

Email: Password:

Login

Switch to Signup

Signup

Email: Password:

Confirm Password:

Signup

Switch to Login

Components

A component is the active part.

- stores the state
`mode = login or signup?`
- responds to user actions such as clicks and form submissions,
`onSubmit()`
- validates input,
`before sending a signup request, check that the signup data has a certain form is present`
- calls services,
`provides functions to make requests/receive responses...`
- sends HTTP requests, receives responses, updates its state,
`... using which makes actual HTTP requests`
- decides when to navigate to another route.
`after successful authentication go to HabitComponent/View`
- also contains the logic that determines what data should be shown.
`mode: 'login' | 'signup' = 'login';`

auth.ts

→ .css / .html / .ts

```
1 import { Component } from '@angular/core';
2 import { Router } from '@angular/router';
3 import { CommonModule } from '@angular/common';
4 import { FormsModule } from '@angular/forms'; //
5 import { AuthService, AuthResponse } from './auth.service';
6
7 @Component({
8   selector: 'app-auth',
9   standalone: true,
10  imports: [CommonModule, FormsModule],
11  templateUrl: './auth.html',
12  styleUrls: ['./auth.css']
13 })
14 export class AuthComponent {
```

Login

Email: Password:

Login

Switch to Signup

Signup

Email: Password:

Confirm Password:

Signup

Switch to Login

./angular_app/node_modules/@angular/router

npm Packages

Component is a decorator function supplied by Angular. @Component is decorator syntax

use as a decorator → @Component({...})
function name → Apply the decorator function Component({...}) to the class AuthComponent

```
@Component({...})  
class AuthComponent {  
  }  
}
```

```
const componentDecorator = Component({...});  
componentDecorator(AuthComponent);
```

```
function componentDecorator(AuthComponent) {  
  AuthComponent.angularInformation = {  
    selector: "app-auth",  
    templateUrl: "./auth.html"  
  };  
  
  return AuthComponent;  
}
```

Turns a TypeScript class into TypeScript class + Angular Info

The selector `app-auth` means that whenever Angular encounters the element `</app-auth>` in a component template, it renders the view represented by `AuthComponent` at that location.

The imports tell Angular to make the template facilities provided by these modules *ngIf, *ngFor, [(ngModel)] and (ngSubmit) available inside this component's HTML template.

auth.ts

```
14 export class AuthComponent {
15   mode: 'login' | 'signup' = 'login';
16
17   loginData = {
18     email: '',
19     password: '',
20   };
21   signupData = {
22     email: '',
23     password: '',
24     confirmPassword: '',
25   };
26   authResponse: AuthResponse | null = null;
27   errorMessage: string | null = null;
28
29   constructor(private authService: AuthService,
30     private router: Router) {}
31 }
```

At what point is the constructor invoked?

.

When it creates an instance of that component.

For example, when the user navigates to localhost:4200/auth

Route or selector requires AuthComponent

Angular resolves AuthService and Router

Angular invokes the AuthComponent constructor

Angular sets up the component and its template

```
export interface AuthResponse {
  token: string;
  user?: {
    id: number;
    email: string;
  };
}
```

Login

Email: Password:

Login

Switch to Signup

auth.ts

0

```
onSubmit(): void {
  this.errorMessage = null;
  if (this.mode === 'login') {
    this.authService.login(this.loginData).subscribe({
      next: (response) => {
        this.authResponse = response;
        localStorage.setItem('token', response.token); // Store token
        console.log('Login successful:', response);
        this.router.navigate(['/habits']);
      },
      error: (err) => {
        this.errorMessage = 'Login failed. Please try again.';
        console.error(err);
      },
    });
  } else {
    // Basic check: ensure password and confirmPassword match
    if (this.signupData.password !== this.signupData.confirmPassword) {
      this.errorMessage = "Passwords don't match.";
      return;
    }
    this.authService.signup(this.signupData).subscribe({
      next: (response) => {
        this.authResponse = response;
        localStorage.setItem('token', response.token); // the backend
        console.log('Signup successful:', response);
        this.router.navigate(['/dashboard']);
      },
      error: (err) => {
        this.errorMessage = 'Signup failed. Please try again.';
        console.error(err);
      },
    });
  }
}
```

Login

Email: Password:

Login

Switch to Signup

Signup

Email: Password:

Confirm Password:

Signup

Switch to Login

set by the view

`this.signupData.password !== this.signupData.confirmPassword`

Only during signup and login, the backend works with the password.

`this.authService.login(this.loginData).subscribe({`

On signup, the backend hashes the password and stores only the hash.

On login, it compares the supplied password with the stored hash. If the login succeeds, it issues a token.

After that, protected components such as Habit and Streak do not resend the password. They send the token with each backend request.

auth.services.ts

```
1 // src/app/auth/auth.service.ts
2 import { Injectable } from '@angular/core';
3 import { HttpClient } from '@angular/common/http';
4 import { Observable } from 'rxjs';
5
6 export interface AuthResponse {
7   token: string;
8   user?: {
9     id: number;
10    email: string;
11  };
12 }
13
14 @Injectable({
15   providedIn: 'root'
16 })
17 export class AuthService {
18   private apiUrl = 'http://localhost:8000/api/auth'; // Adjust to your backend's URL
19
20   constructor(private http: HttpClient) {}
21
22   // login takes a credentials object with email and password properties and returns an
23   // Observable of type AuthResponse. To do this it sends a POST request to the
24   // /login endpoint of the backend API.
25   login(credentials: { email: string; password: string })
26     : Observable<AuthResponse> {
27     return this.http.post<AuthResponse>(`${this.apiUrl}/login`, credentials);
28   }
29
30   signup(details: { email: string; password: string; confirmPassword: string })
31     : Observable<AuthResponse> {
32     return this.http.post<AuthResponse>(`${this.apiUrl}/signup`, details);
33   }
34 }
```

POST /api/auth/login HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Accept: application/json

{
 "email": "amit@gmail.com",
 "password": "something"
}

@Injectable({ providedIn: 'root' })
tells Angular that AuthService is an injectable service. Available throughout the application.

One instance shared by all components.

Usage

```
constructor(private authService: AuthService) {}
```

High level meaning of login ():
Send the email and password to the backend login endpoint, and return an observable that will later provide the authentication response.

1. subscribe() starts the HTTP request. Also registers continuation.
2. The current call stack continues and eventually finishes.
3. The browser handles the network request independently.
4. When the response arrives, an event is placed in the event queue.
5. The event loop later invokes next(response).
6. Execution continues inside that callback.

```

2 <div class="auth-container">
3   <h2 *ngIf="mode === 'login'">Login</h2>
4   <h2 *ngIf="mode === 'signup'">Signup</h2>
5
6   <form (ngSubmit)="onSubmit()">
7     <div *ngIf="mode === 'login'">
8       <label for="email">Email:</label>
9       <input type="email" [(ngModel)]="loginData.email" name="email" required />
10      &nbsp;&nbsp;&nbsp;
11      <label for="password">Password:</label>
12      <input type="password" [(ngModel)]="loginData.password" name="loginPassword" required />
13    </div>
14    <div *ngIf="mode === 'signup'">
15      <label for="signupEmail">Email:</label>
16      <input type="email" [(ngModel)]="signupData.email" name="signupEmail" required />
17      &nbsp;&nbsp;&nbsp;
18      <label for="signupPassword">Password:</label>
19      <input type="password" [(ngModel)]="signupData.password" name="signupPassword" required />
20    </div>
21    &nbsp;&nbsp;&nbsp;
22    <div *ngIf="mode === 'signup'">
23      <label for="confirmPassword">Confirm Password:</label>
24      <input id="confirmPassword" type="password" [(ngModel)]="signupData.confirmPassword" name="confirmPassword"
25      required />
26    </div>
27    &nbsp;&nbsp;&nbsp;
28    <div>
29      <button type="submit">{{ mode === 'login' ? 'Login' : 'Signup' }}</button>
30      &nbsp;&nbsp;&nbsp;
31      <button type="button" (click)="toggleMode()">
32        Switch to {{ mode === 'login' ? 'Signup' : 'Login' }}
33      </button>
34    </div>
35
36    <div *ngIf="errorMessage" class="error">{{ errorMessage }}</div>
37
38    <div *ngIf="authResponse" class="success">
39      <p>Authentication Successful!</p>
40    </div>
41  </form>
42 </div>

```

conditional <h2>

automatic binding from DOM to listeners

Two way communication from view to component

logindata

signupdata

submit/toggle

error/success message

Login

Email: Password:

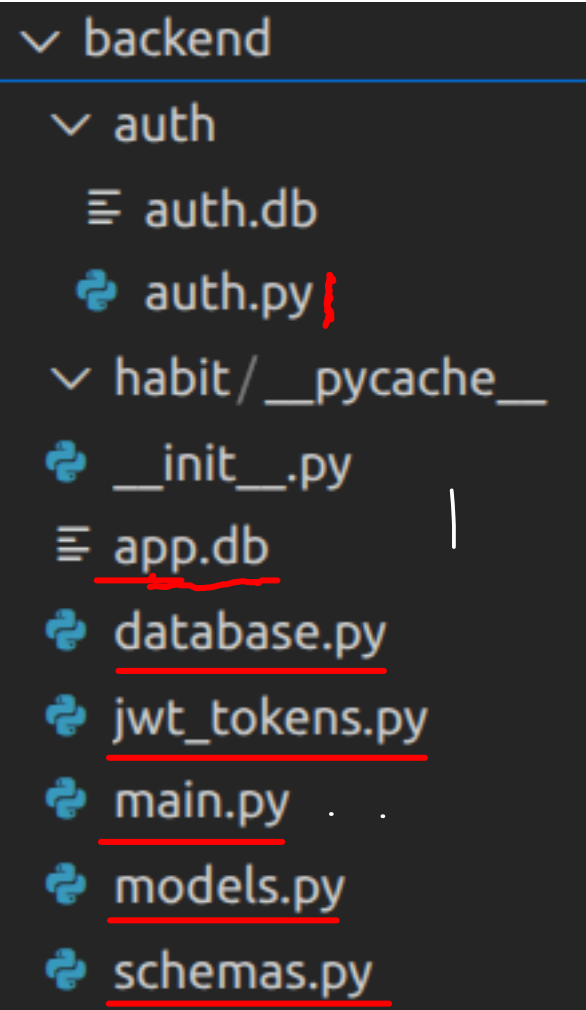
Signup

Email: Password:

Confirm Password:

SQLAlchemy } Objects in Python Backend

~ ap



```
1
2 from fastapi import FastAPI
3 from models import User
4 from database import engine, Base
5 from auth.auth import router as auth_router
6 from fastapi.middleware.cors import CORSMiddleware
7 Base.metadata.create_all(bind=engine)
8 app = FastAPI()
9
10 app.add_middleware(
11     CORSMiddleware,
12     allow_origins=["http://localhost:4200"],
13     allow_methods=["*"],
14     allow_headers=["*"],
15 )
16
17 app.include_router(auth_router, prefix="/api/auth",
18 tags=["auth"])
19 @app.get("/")
20 def read_root():
21     return {"message": "Backend is running"}
```

main.py: The main file is the entry point of the Python web app(FastAPI in our case).

Entry point of the FastAPI backend. Creates the application, connects the routers, enables communication with Angular, and defines a small test endpoint.

from models import User

Imports the User database model.

from database import engine, Base

engine, represents the connection to the SQLite database.
Base, from which all SQLAlchemy model classes inherit.

from auth.auth import router as auth_router

Imports the routers defined in the authentication and habit modules.

Imports the CORS middleware, needed because Angular and FastAPI run at different origins
Angular: <http://localhost:4200>, FastAPI: <http://localhost:8000>

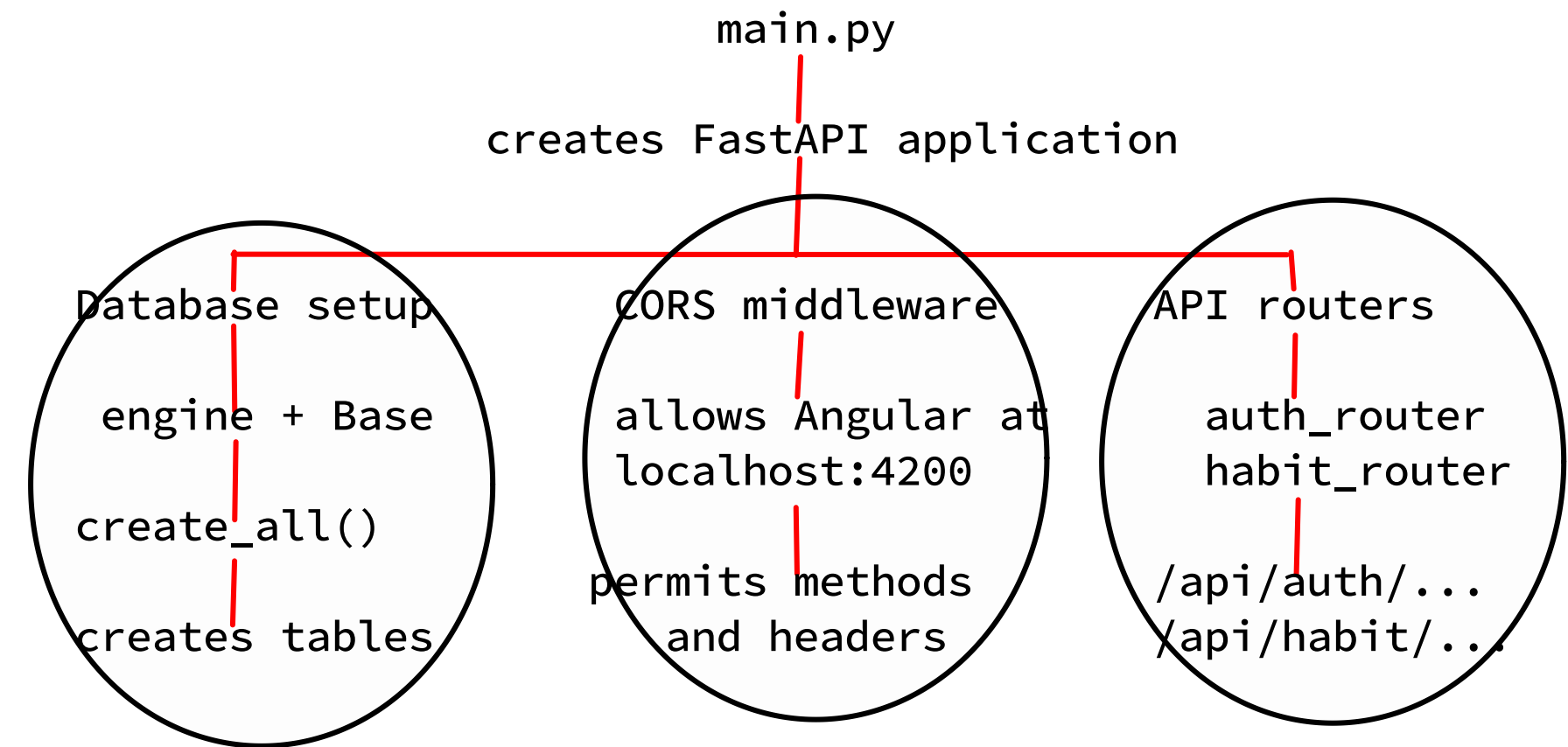
Base.metadata.create_all(bind=engine)
create all known database tables if they do not already exist.

app = FastAPI()
Creates the FastAPI application object app
uvicorn main:app --reload

app.add_middleware(
 CORSMiddleware,...

The three aspects of the backend

- FastAPI creates the overall app
 - CORS allows Angular at 4200 to talk to backend at 8000
 - Database is the persistent storage
 - Router - to respond to Angular requests



auth.py (defines router)

```
1 from fastapi import APIRouter, HTTPException, Depends
2 from models import User
3 from database import engine, Base, get_db
4 from sqlalchemy.orm import Session
5 from passlib.hash import bcrypt
6 from datetime import timedelta
7 from jwt_tokens import create_access_token,
  ACCESS_TOKEN_EXPIRE_MINUTES
8 from schemas import AuthRequest, AuthResponse
9
10
11 Base.metadata.create_all(bind=engine)
12
13 router = APIRouter()
```

auth.py

```
18
19 @router.post("/signup", response_model=AuthResponse)
20 def signup(data: AuthRequest, db: Session =
  Depends(get_db)):
21
22     ... existing_user =
23         db.query(User).filter_by(email=data.email).first()
24     ... if existing_user:
25         ... raise HTTPException(status_code=400,
26             detail="User already exists")
27
28     ... hashed_pw = bcrypt.hash(data.password)
29     ... user = User(email=data.email,
30         hashed_password=hashed_pw)
31     ... db.add(user)
32     ... db.commit()
33     ... db.refresh(user)
34
35     # Create token with user ID as the subject
36     access_token_expires =
37         timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
38     token = create_access_token(
39         data={"sub": str(user.id)},
40         expires_delta=access_token_expires)
41
42     return {"token": token, "user": {"id": user.id,
43         "email": user.email}}
```

If this router receives an HTTP POST request whose path is /signup, call the `signup()` function to handle it.

main.py mounts this router with:

`app.include_router(auth_router, prefix="/api/auth")`

complete endpoint is: POST /api/auth/signup

endpoint = HTTP method + PATH

route = Rule that matches a PATH based event to a handler

Angular: PATH -> Component

FASTAPI: HTTP method + PATH -> function

To FastAPI: "Do not expect the caller to supply db directly. Call `get_db()` and use what it provides."

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

FastAPI creates generator
FastAPI advances generator

`get_db` creates Session
`get_db` yields Session

FastAPI passes Session as route parameter `db`

```
SELECT * from users
where email =
limit = 1
```

`signup` uses db
`signup` finishes

FastAPI closes generator

finally runs
Session is closed

auth.py

```
18
19 @router.post("/signup", response_model=AuthResponse)
20 def signup(data: AuthRequest, db: Session = Depends(get_db)):
21     ... existing_user =
22         db.query(User).filter_by(email=data.email).first()
23     ... if existing_user:
24         ... raise HTTPException(status_code=400,
25                                 detail="User already exists")
26     ... hashed_pw = bcrypt.hash(data.password)
27     ... user = User(email=data.email,
28                     hashed_password=hashed_pw)
29     ... db.add(user)
30     ... db.commit()
31     ... db.refresh(user)
32     ... # Create token with user ID as the subject
33     ... access_token_expires =
34         timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
35     ... token = create_access_token(
36         ... data={"sub": str(user.id)},
37         ... expires_delta=access_token_expires)
38     ... return {"token": token, "user": {"id": user.id,
39                                         "email": user.email}}
```

- o Create a row of the table
- o Register with SQLAlchemy as a new row to be installed in the database
- o Permanently save the row in the database
- o Reload the object from the database (sets the user.id field)

An example token:

```
{"sub": "5";
  "exp": absolute time
  ...
}
```

User - A class with fields as the columns of an underlying database
user - An instance of User. A row of the database
users - The database table

AuthResponse

Other backend files

database.py

```
1
2 from sqlalchemy import create_engine
3 from sqlalchemy.ext.declarative import declarative_base
4 from sqlalchemy.orm import sessionmaker
5
6 DATABASE_URL = "sqlite:///./app.db"
7
8 engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
9 SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
10
11 Base = declarative_base()
12 def get_db():
13     db = SessionLocal()
14     try:
15         yield db
16     finally:
17         db.close()
```

No changes

SQLAlchemy models
-Database tables in terms of Python objects
-SQL operations in terms of Python functions

The database is the file app.db

Sessionlocal () creates a session

A database engine can create connections

Base: Base class from which other database classes are derived

models.py

```
1 from sqlalchemy import Column, Integer, String, Boolean, Date
2 from database import Base
3
4 class User(Base):
5     __tablename__ = "users"
6
7     id = Column(Integer, primary_key=True, index=True)
8     email = Column(String, unique=True, index=True)
9     hashed_password = Column(String)
10
```

modify

Now you know where the User class was defined

schemas.py

```
1 from pydantic import BaseModel
2 from datetime import date
3
4 # Schemas          modify
5
6 class AuthRequest(BaseModel):
7     email: str
8     password: str
9
10 class UserOut(BaseModel):
11     id: int
12     email: str
13
14 class AuthResponse(BaseModel):
15     token: str
16     user: dict
```

Pydantic lets you define data models for validation and serialization

```
class AuthRequest(BaseModel):
    email: str
    password: str
```



```
{
    "email": "amit@gmail.com",
    "password": "something"
}
```

jwt-tokens.py

Need not be changed
Have a look at:
 create_access_token()
to understand the
structure of JWT tokens

What you have to do - HabitComponent

0. habit.html

Should generate the following view

Add / Edit Habits

Habit name Daily

- 1 — Yoga — Daily
- 2 — Drink Water — Daily
- 3 — Run — Daily
- 4 — Cycling — Daily
- 5 — Dance — Weekly

What you have to do – HabitComponent

1. habit.html

Since I have shown you the UI, you should know what has to be done. Use a form to accept the habit name and frequency, and bind the input fields to properties of a newHabit object using [(ngModel)]. Handle form submission with (ngSubmit) and call a component method that sends the new habit to the backend.

Display the existing habits as a list using *ngFor, so Angular repeats one element for every object in the habits array. Inside each repeated list item, use {{ }} to show values such as the habit ID, name, and frequency. Add buttons whose click handlers receive the current habit's ID, allowing the component to mark that habit complete or delete it. The completed template should therefore connect user input, component state, repeated display, and event handling in one view.

2. habit.ts (should use the services in habit.service.ts)

Should implement:

```
ngOnInit(): void # HabitComponent should implement the interface
OnInit which requires it to define the function ngOnInit()
getHabits(): void
addHabit(): void
deleteHabit(habitId: number): void
markComplete(habitId: number): void
```

3. habit.service.ts

Should implement:

```
private getAuthHeaders(): HttpHeaders # Returns the Authorization
header stored in the browser localStorage so that the backend can
authenticate requests from HabitComponent.
getHabits(): Observable<Habit[]>
addHabit(habitData: Partial<Habit>): Observable<Habit>
deleteHabit(habitId: number): Observable<unknown>
markComplete(habitId: number, completionData: HabitCompletionIn)
: Observable<unknown>
getCompletions(habitId: number): Observable<HabitCompletionOut[]>
```

3. habit.py

What you have to do – HabitComponent

You may have to modify main.py models.py and schemas.py. To implement the router in habit.py, you have to provide the routes for the following endpoints:

GET "/"

```
def getHabits(db: Session = Depends(get_db), current_user: int = Depends(get_user_id))
```

POST "/"

```
def create_habit(habit_data: HabitCreate, db: Session = Depends(get_db),  
                 current_user: int = Depends(get_user_id))
```

DELETE("/{habit_id}")

```
def delete_habit(habit_id: int, db: Session = Depends(get_db),  
                 current_user: int = Depends(get_user_id))
```

POST("/{habit_id}/complete"

```
def mark_complete(habit_id: int, data:HabitCompletionIn, db: Session = Depends(get_db),  
                  current_user:int = Depends(get_user_id)):
```